

Yapay Zekâ Destekli Yazılım Geliştirmede Teknik Borç: Sınırlar, Riskler ve Yapısal Bileşenler

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Yazılım geliştirme süreçlerine yapay zekâ destekli araçların dahil olması, üretim hızını belirgin biçimde artırmıştır. Ancak bu dönüşüm, yazılım mühendisliğinin köklü problemlerinden biri olan teknik borç kavramını ortadan kaldırmamış; aksine bu kavramın niteliğini ve stratejik önemini değiştirmiştir. Bu çalışma, yapay zekâ destekli geliştirme ile teknik borç arasındaki ilişkiyi incelemekte; teknik borcun temel bileşenlerini tanımlamakta ve bu borcun yapay zekâ sistemlerinin üretkenliği üzerindeki sınırlayıcı etkilerini tartışmaktadır.

Temel tez şudur: Yapay zekâ, teknik borcu yüksek bir kod tabanı üzerinde de üretim yapabilir; fakat belirli bir eşikten sonra teknik borç, yalnızca insan geliştiricilerin değil, yapay zekâ sistemlerinin de doğruluğunu, hızını ve güvenilirliğini sınırlamaya başlar. Bu nedenle teknik borç, yapay zekâ çağında yalnızca bir bakım maliyeti değil, insan-makine ortak üretim kapasitesini belirleyen stratejik bir değişken haline gelmiştir.

Yapay Zekânın Kod Üretim Mekanizması ve Teknik Borç

Yapay zekâ destekli geliştirme araçları, mevcut kod tabanındaki örüntülerden, soyutlamalardan ve mimari tercihlerden hareketle öneriler üretir. Bu mekanizma, kod tabanının kalitesi ile üretilen kodun kalitesi arasında doğrudan bir bağımlılık oluşturur. Kod tabanında yanlış soyutlamalar, tutarsız yapılar, gizli bağımlılıklar ve kırılgan akışlar bulunduğunda, yapay zekâ bu problemleri çoğu zaman düzelterek değil, onları temel alarak üretim yapar.

Bu nedenle yapay zekâ, teknik borçlu bir sistemde ilk aşamada yüksek üretim hızı sağlıyor gibi görünebilir. Ancak zamanla mevcut bozuk yapının üzerine yeni katmanlar eklenir ve sistemin toplam karmaşıklığı artar. İlk hız yanıltıcıdır; çünkü üretimin hızı ile üretimin sistem düzeyindeki doğruluğu birbirinden farklı ölçütlerdir.

Yanlış Soyutlamaların Öğrenilmesi

Yapay zekâ, kod üretirken mevcut projedeki yapılara referans verir. Projede yanlış veya zayıf soyutlamalar bulunuyorsa, model bunları sistemin geçerli mimarisi olarak algılayabilir.

Örneğin bir bileşen gereğinden fazla sorumluluk taşıyorsa, yapay zekâ yeni bileşenleri de benzer şekilde tasarlama eğilimi gösterir. Bir servis aynı anda dış sistem çağırısı yapıyor, veri

dönüştürüyor, küresel durumu değiştiriyor ve arayüz davranışını etkiliyorsa, model bu yapıyı projenin standardı olarak kabul edebilir. Bu durumda yapay zekâ yalnızca kod üretmekle kalmaz; mevcut mimari hataları da yeniden üretir.

Kötü Örüntülerin Çoğaltılması

Teknik borçlu sistemlerde kötü örüntüler genellikle tek bir noktada kalmaz; zamanla tekrar edilir, kopyalanır ve sistemin farklı bölgelerine yayılır. Yapay zekâ, kod tabanındaki tekrar eden örüntüleri güçlü sinyaller olarak okur. Dolayısıyla kötü bir örüntü sistemde ne kadar yaygınsa, modelin onu “doğru proje standardı” olarak yorumlama olasılığı o kadar artar.

Bu durumun sonucu şu şekilde ifade edilebilir:

Model, mevcut bozuk yapıyı temizlemek yerine, aynı yapının yeni örneklerini üretir.

Bu mekanizma, teknik borcun yalnızca varlığını sürdürmesine değil, daha yüksek bir hızla yayılmasına neden olabilir.

Regression Riski

Teknik borcu yüksek sistemlerde parçalar arasındaki ilişkiler açık değildir. Bir değişikliğin hangi ekranı, servisi, durum alanını veya kullanıcı akışını etkileyeceği kolayca öngörülemez. Bu koşullarda yapay zekâ tarafından yapılan bir değişiklik, yerel düzeyde doğru görünse bile sistemin başka bir bölgesinde beklenmeyen hatalara, yani regression'lara yol açabilir.

Regression riski özellikle şu koşullarda artar:

- sistemde gizli bağımlılıkların bulunması,
- test kapsamının zayıf olması,
- iş kurallarının açıkça belgelenmemiş olması,
- aynı verinin farklı bağlamlarda farklı anlamlarla kullanılması,
- durum akışlarının belirsiz olması,
- geçmiş sınır durumlarının kod içinde örtük biçimde gömülü kalması.

Bu nedenle yapay zekâ ile üretilen kodun güvenilirliği, yalnızca modelin kapasitesine değil, kod tabanının anlaşılabilirliğine de bağlıdır.

Bağlam Penceresi Sınırı

Yapay zekâ modelleri sınırlı bir bağlam penceresi içinde çalışır; yani aynı anda yalnızca belirli miktarda kodu, dokümantasyonu ve geçmiş bilgiyi dikkate alabilir. Teknik borcu düşük bir sistemde bu sınırlılık büyük bir problem oluşturmaz; çünkü sistemin sınırları, sorumlulukları ve ilişkileri görece nettir. Ancak teknik borcu yüksek bir sistemde gerçek davranış birçok dosyaya, geçmiş kararlara, gizli durumlara ve dolaylı bağımlılıklara yayılmıştır.

Bu koşullarda modelin bağlam penceresi yetersiz kalır ve değiştirilen kodun sistemin diğer bölgeleriyle ilişkisi tam olarak görülemez. Sonuç şu şekilde özetlenebilir:

Model, yerel düzeyde doğru görünen değişiklikler üretebilir; fakat sistem düzeyindeki etkileri gözden kaçırabilir.

Değişikliklerin Etki Alanının Belirsizliği

Teknik borçlu sistemlerde değişikliklerin etki alanı öngörülemezdir. Küçük bir güncelleme, beklenmedik biçimde başka bir modülü bozabilir. Örneğin bir fonksiyondaki veri formatı değişikliği yalnızca o fonksiyonu değil; ona bağlı bileşenleri, testleri, önbellekleme mekanizmasını, istemci tarafındaki kalıcı kayıtları, sunucu tarafındaki veri eşlemesini veya kullanıcı akışlarını da etkileyebilir.

Bu durum insan geliştirici için de zorlayıcıdır; ancak yapay zekâ için daha kritiktir. Çünkü model, sistemin tarihsel bağlamını ve organizasyonel hafızasını doğal olarak taşımaz; kendisine açıkça verilmeyen bağımlılıkları göremez. Dolayısıyla teknik borç arttıkça, modelin yaptığı değişikliklerin güvenilirliği azalır.

Test Güvenilirliği ve Geri Bildirim Mekanizması

Yapay zekâ destekli geliştirmede testler yalnızca bir kalite güvence aracı değildir; aynı zamanda modelin temel geri bildirim mekanizmasıdır. Güvenilir testler mevcutsa, model yaptığı değişikliğin sistemi bozup bozmadığını daha net değerlendirebilir. Ancak testler eksik, kırılgan, yüzeysel veya güncel değilse, model sağlıklı geri bildirim alamaz.

Bu koşullarda model şu soruları yanıtlayamaz:

- yapılan değişiklik gerçekten doğru mu,
- yalnızca mevcut testler mi geçirildi,
- kritik bir kullanıcı akışı bozuldu mu,
- geçmiş bir sınır durumu ihlal edildi mi,
- sistem davranışı beklendiği gibi korundu mu.

Testlerin güvenilir olmadığı sistemlerde model, bir noktadan sonra işlevsel olarak körleşir: kod üretmeye devam eder, fakat ürettiği kodun sistem üzerindeki gerçek etkisini değerlendiremez.

Yapay Zekâ Teknik Borcu Ortadan Kaldırmaz

Yapay zekâ, teknik borcu otomatik olarak çözmez. Teknik borç; mimari, bağlam, sorumluluk, bağımlılık ve karar hafızası problemidir. Yapay zekâ bu alanların her birinde yardımcı olabilir; fakat sistemin yapısal problemleri çözülmeden yalnızca yeni kod üretmek çoğu zaman yeterli değildir.

Teknik borcun yüksek olduğu koşullarda yapay zekâ genellikle iki sonuçtan birini üretir:

- teknik borcu geçici olarak maskeler,
- teknik borcun daha hızlı büyümesine neden olur.

Maskeleme etkisi, kısa vadede yeni özelliklerin eklenmiş görünmesinden kaynaklanır. Büyütme etkisi ise mevcut bozuk örüntülerin üzerine yeni kod katmanlarının eklenmesinden doğar. Bu nedenle yapay zekâ ile geliştirilen sistemlerde teknik borç yönetimi önemini kaybetmez; aksine daha kritik hale gelir.

Teknik Borcun Temel Bileşenleri

Teknik borç, yalnızca kötü yazılmış kod anlamına gelmez. Daha kapsayıcı bir tanımla teknik borç; sistemin iç yapısında biriken tutarsızlıkların, belirsizliklerin, gizli bağımlılıkların ve kontrolsüz yan etkilerin toplamıdır. Bu borç arttıkça sistemin davranışını anlamak, değiştirmek, test etmek ve güvenle geliştirmek zorlaşır.

Tutarsızlık

Tutarsızlık, sistemin farklı bölgelerinde aynı problemin farklı yöntemlerle çözülmesi durumudur. Bir modülde belirli bir mimari kural uygulanırken başka bir modülde farklı bir yaklaşım kullanılıyorsa, sistemin zihinsel modeli bozulur.

Örneğin hata yönetimi bir modülde merkezi bir yardımcı yapı üzerinden yürütülürken, başka bir modülde doğrudan yerel yakalama bloklarıyla, bir diğerinde ise sessizce yutulan hatalarla ele alınıyorsa, sistem tutarsız hale gelmiş demektir. Geliştiricinin “bu projede doğru yöntem hangisi” sorusunu sık sorması, teknik borcun yalnızca kod düzeyinde değil, mimari anlayış düzeyinde de biriktiğinin göstergesidir.

Gizli Bağımlılık

Gizli bağımlılık, bir parçanın başka bir parçaya açıkça görünmeyen biçimde bağlı olmasıdır. Kod ilk bakışta bağımsız görünür; fakat gerçekte belirli bir çalışma sırası, belirli bir durum, belirli bir ortam değişkeni, belirli bir sunucu yanıtı veya önceki bir işlemin yan etkisi olmadan çalışmaz.

Örneğin bir bileşen, doğrudan parametre alıyor gibi görünürken, arka planda küresel durumdaki eski bir değere, istemci tarafında kalıcı olarak saklanmış bir kayda veya daha önce yürütülmüş bir işlemin yan etkisine bağımlı olabilir. Bu durumda bağlantılar görünmez hale gelir ve geliştirici neyin neye bağlı olduğunu izleyemez.

Kırılğan Akış

Kırılğan akış, sistemin yalnızca belirli koşullar altında doğru çalışması durumudur. Akışın küçük bir parçası değiştiğinde beklenmeyen noktalar bozuluyorsa, akış kırılğandır.

Örneğin kullanıcı belirli bir ekrana yalnızca belirli bir sıradan geçerek ulaştığında sistem doğru çalışıyor, fakat doğrudan erişimde durum eksik kalıyorsa; bir sunucu yanıtı geciktiğinde yükleme göstergesi kapanmıyorsa; bir düğmeye ardışık iki kez basıldığında yinelenen kayıt oluşuyorsa akış kırılğandır. Kırılğan akışların ortak özelliği şudur:

Sistem çalışıyor görünür; fakat güvenilir değildir.

Bu tür akışlarda problem çoğu zaman sözdizimsel değildir; akışın tasarımında yeterli sınırların ve garantilerin bulunmamasıdır.

Belirsiz Mimari

Belirsiz mimari, sistemin hangi parçalarının hangi sorumluluklara sahip olduğunu net olmaması durumudur. Bir bileşen aynı anda arayüz çiziyor, dış sistem çağrısı yapıyor, durumu değiştiriyor, doğrulama yürütüyor ve iş kuralı çalıştırıyorsa mimari bulanıklaşır.

Bu durumda sistemde şu sorumluluk ayrımları kaybolur:

- arayüz sorumluluğu,
- durum yönetimi sorumluluğu,
- dış sistem iletişimi sorumluluğu,
- iş kuralı sorumluluğu,
- hata yönetimi sorumluluğu,
- veri dönüştürme sorumluluğu.

Mimari belirsizleştğinde yeni özellik eklemek zorlaşır; çünkü geliştirici kodu nereye yerleştireceğini bilemez. Bu belirsizlik zamanla rastgele eklemelere, tekrar eden çözümlere ve daha fazla teknik borca yol açar.

Eski Varsayımlar

Eski varsayımlar, sistemin geçmişte geçerli olan fakat artık geçerliliğini yitirmiş kabuller üzerinde çalışmaya devam etmesidir. Örneğin başlangıçta tek kullanıcı için tasarlanmış bir sistem çok kullanıcıya hale gelmiş, fakat veri modeli hâlâ tek kullanıcı varsayımını taşıyor olabilir. Benzer biçimde, başlangıçta tek bir veri formatını destekleyen bir sistem zamanla farklı format ve girdi türlerini desteklemeye başlamış, fakat eski servisler ilk formatın varsayımlarıyla yazılmış olabilir.

Eski varsayımların tehlikesi, kodda açıkça görünmemelerinden kaynaklanır; sistemin davranışını içeriden ve sessizce belirlerler.

KontROLSÜZ Yan Etkiler

Yan etki, bir işlemin kendi ana sonucunun dışında sistemde başka bir değişiklik üretmesidir. Her yan etki zararlı değildir; ancak kontROLSÜZ, belirsiz ve rastgele yan etkiler ciddi teknik borç üretir.

Örneğin yalnızca veri getirmesi beklenen bir fonksiyon, aynı zamanda küresel durumu değiştiriyor, istemci tarafında kalıcı kayıt yazıyor, başka bir olay tetikliyor veya arayüzde bildirim gösteriyorsa artık saf değildir; çağrılması sistemde öngörülemeyen sonuçlar doğurur. Bu koşullarda geliştiricinin karşılaştığı temel risk şudur: yalnızca bir fonksiyon çağrılmıştır, fakat sistemin başka bir bölgesi bozulmuştur. KontROLSÜZ yan etkiler sistemin tahmin edilebilirliğini azaltır ve test yazımını zorlaştırır; çünkü tek bir davranış test edilirken başka davranışlar da tetiklenir.

Karar Hafızasının Kaybı

Teknik borcun en tehlikeli biçimlerinden biri, bir kodun neden o şekilde yazıldığının artık bilinmemesidir. Kod çalışıyor olabilir; fakat arkasındaki karar kaybolmuştur. Bu durumda ekip,

yapının gerçekten gerekli olup olmadığını, hangi hatayı engellediğini veya hangi geçmiş ihtiyaca yanıt verdiğini bilmediği için kodu değiştirmekten kaçınır.

Bu tür kodlar sistemde dokunulmaz bölgeler oluşturur. Bir projede “buna dokunmayalım, bir şeyleri bozabilir” ifadesi sık duyuluyorsa, teknik borç artık yalnızca teknik bir problem değil, organizasyonel hafıza problemine de dönüşmüştür.

Teknik Borcun Yapay Zekâ Destekli Süreçte Görünür Hale Gelmesi

Yapay zekâ destekli geliştirme süreçlerinde teknik borcun etkisi başlangıçta çoğu zaman hissedilmez; hatta ilk aşamada model, borçlu sistemlerde bile yüksek üretim hızı sağlayabilir. Yeni özellikler hızla eklenir, tekrar eden kodlar hızla üretilir ve ekip belirgin bir verimlilik artışı yaşadığını düşünebilir.

Ancak sistem büyüdükçe bu ilk hızın sınırları belirginleşir. Kod tabanı karmaşıklaştıkça, mimari tutarsızlıklar arttıkça ve gizli bağımlılıklar çoğaldıkça modelin performansı düşmeye başlar. Bu aşamada tipik olarak şu belirtiler gözlemlenir:

- modele verilen talimatlar giderek uzar,
- bağlam yönetimi karmaşıklaşır,
- aynı hata tekrar tekrar ortaya çıkar,
- model eski ve kötü örüntüleri kopyalar,
- regression sayısı artar,
- küçük değişiklikler beklenmeyen bölgeleri bozar,
- değişikliklerin etki alanı öngörülemez hale gelir.

Bu tablo, teknik borcun arka planda duran bir kalite problemi olmaktan çıkıp, yapay zekâ destekli geliştirme sürecinin verimliliğini doğrudan sınırlayan bir engel haline geldiğini gösterir.

Yapay Zekâ Performansını Düşüren Teknik Borç Türleri

Yapay zekâ, kod tabanındaki düzenli örüntülerden, isimlendirme standartlarından, veri akışlarından ve mimari sınırlardan faydalanarak öneri üretir. Dolayısıyla sistem ne kadar tutarlıysa model o kadar etkili çalışır. Buna karşılık aşağıdaki koşullarda model performansı belirgin biçimde düşer:

- aynı durumun birden fazla kaynağının bulunması,
- isimlendirme standardının tutarsız olması,
- örtük yan etkilerin varlığı,
- bileşen sorumluluklarının belirsizliği,
- iş kurallarının arayüz katmanına gömülmesi,

- testlerin kararsız davranması,
- aynı işin birden fazla farklı gerçekleştirimle yapılması,
- geçici çözümlerin kalıcı hale gelmesi,
- veri akışının izlenememesi.

Bu problemlerin ortak noktası, sistemin okunabilirliğini ve tahmin edilebilirliğini azaltmaktır. İnsan geliştirici için zorlayıcı olan bu koşullar, yapay zekâ için daha da kısıtlayıcıdır; çünkü model, sistemin gerçek niyetini değil, büyük ölçüde mevcut kod yapısından çıkardığı örüntüleri izler.

Tekil Doğruluk Kaynağının Yokluğu

Aynı verinin sistemde birden fazla yerde, farklı biçimlerde tutulması hem insan hem de model için temel bir belirsizlik üretir: gerçek kaynak hangisidir? Bu sorunun yanıtı net değilse, model yanlış durum alanını güncelleyebilir, kullanılmayan bir alanı düzeltebilir veya gerçek kullanıcı akışını etkilemeyen bir değişiklik yapabilir. Bu nedenle tekil doğruluk kaynağı ilkesi, yapay zekâ destekli geliştirmede daha da kritik hale gelir.

Tutarsız İsimlendirme

İsimlendirme, kod tabanının anlam sistemidir. Aynı kavramın farklı bölgelerde farklı adlarla temsil edilmesi, bunların gerçekten farklı kavramlar mı yoksa aynı alan nesnesinin farklı adları mı olduğu konusunda belirsizlik üretir. Model bu belirsizlikte yanlış eşleştirmeler yapabilir. Dolayısıyla tutarlı isimlendirme yalnızca okunabilirlik sağlamaz; model için anlam kararlılığı üretir.

Kararsız Testler

Kararsız test, aynı koşullarda bazen geçen bazen başarısız olan testtir; güvenilir geri bildirim üretmez. Testler güvenilir değilse, “test başarısız olduğunda gerçekten kod mu bozuk, yoksa test mi güvenilirmez” sorusu yanıtlanamaz. Bu belirsizlik hem insan geliştiriciyi hem de modeli yavaşlatır ve geri bildirim döngüsünün değerini düşürür.

Geçici Çözümlerin Kalıcılığı

Geçici çözümler çoğu zaman hız kazanmak amacıyla eklenir; ancak belgelenmez, testle güvence altına alınmaz ve zamanında temizlenmezse kalıcı teknik borca dönüşür. Model, böyle bir çözümün neden var olduğunu bilemez; onu gereksiz görüp kaldırabilir ya da tersine, doğru örüntü sanıp sistemin diğer bölgelerine yayabilir. Bu nedenle geçici çözümler açıkça işaretlenmeli ve izlenebilir kayıtlarla takip edilmelidir.

Sektörel Kırılma: Kötü Kodun Yeni Maliyeti

Yazılım sektöründe önemli bir kırılma yaşanmaktadır. Geçmişte kötü kodun temel sonucu insan geliştiricinin yavaşlamasıydı: kod tabanı karmaşılaştıkça geliştirici daha fazla zaman harcar,

daha çok hata yapar ve deęişikliklerden kaçınırdı. Günümüzde ise kötü kod yalnızca insanı deęil, yapay zekâ destekli geliştirme sistemlerini de yavaşlatmaktadır.

Bu farkın stratejik anlamı şudur:

Geçmişte kötü kod, insan geliştiricinin üretkenliğini düşürüyordu. Bugün kötü kod, yapay zekâ destekli geliştirme kapasitesinin doğruluğunu, hızını ve güvenilirliğini de düşürmektedir.

Geçmişte teknik borcun temel sorusu “insan geliştirici bu kodu anlayabiliyor mu” iken, bugün bu soru genişlemiştir: “insan ve yapay zekâ bu sistem üzerinde birlikte akıl yürütebiliyor mu?” Modern yazılım geliştirmede verimlilik artık yalnızca insanın kodu tek başına anlamasına deęil, insan ile yapay zekânın ortak çalışma kapasitesine bağlıdır.

Sonuç

Yapay zekâ, yazılım üretim hızını artıran güçlü bir araçtır; ancak teknik borcu ortadan kaldırmaz. Aksine, teknik borçlu sistemlerde ya borcu geçici olarak maskeler ya da onun daha hızlı büyümesine aracılık eder. Bunun nedeni, modelin mevcut kod tabanındaki örüntüleri temel almasıdır: bozuk örüntüler, model için yanlış öğrenme sinyalleri üretir.

Teknik borcun bileşenleri — tutarsızlık, gizli bağımlılık, kırılgan akış, belirsiz mimari, eski varsayımlar, kontrolsüz yan etkiler ve karar hafızasının kaybı — yalnızca insan üretkenliğini deęil, yapay zekâ destekli üretim kapasitesini de sınırlar. Bağlam penceresi sınırlılığı ve test güvenilirliğine bağımlılık, bu sınırlamayı daha da belirginleştirir.

Sonuç olarak yapay zekâ çağında teknik borç, klasik anlamıyla bir bakım maliyeti olmaktan çıkmış; insan–makine ortak akıl yürütme kapasitesini belirleyen stratejik bir engel haline gelmiştir. Bu nedenle yapay zekâ ile geliştirilen sistemlerde teknik borç yönetimi, önemini yitirmek bir yana, geliştirme stratejisinin merkezî unsurlarından biri olarak ele alınmalıdır.